



ESCUELA DE INGENIERÍA Y CIENCIAS

INGENIERÍA EN CIENCIAS DE DATOS Y MATEMÁTICAS

Gestor de Identidades

Documentación de Desarrollo — Versión 1.0

MA2006B — Uso de Álgebras Modernas para Seguridad y Criptografía

Elaborado por:

Nombre	Matrícula
Ana Laura Reyes Alvarez	A00838315
Nicolás Romero Flota	A01770163
Santiago Pérez Almazán	A01800790
Jose Emilio Gomez Santos	A01665895
Marcelo Sánchez Rodríguez	A01769740
Jesús Emilio Medina García	A00839492

Equipo: 3 | **Grupo:** 604

Profesores:

Daniel Otero | Anas Wajid | Alberto F. Martínez

Socio Formador:

Programa de Atención a Migrantes y sus Familias “Casa Monarca”

Índice general

1. Descripción General	1
1.1. Propósito del sistema	1
1.2. Alcance	1
1.3. Casos de uso principales	1
2. Arquitectura del Sistema	3
2.1. Tipo de arquitectura	3
2.2. Componentes principales	3
2.3. Diagrama general	3
3. Stack Tecnológico	5
3.1. Justificación del stack	5
4. Estructura del Proyecto	6
4.1. Organización de carpetas	6
4.2. Descripción de módulos	6
5. Configuración del Entorno	7
5.1. Requisitos y versiones	7
5.2. Variables de entorno	7
5.3. Archivos de configuración	8
6. Instalación y Ejecución	9
6.1. Pasos para levantar el sistema	9
6.2. Comandos principales	9
7. Base de Datos	10
7.1. Modelo de datos	10
7.2. Tablas principales	10
7.3. Relaciones	11
8. API / Interfaces	12
8.1. Convenciones generales	12
8.2. Interfaces principales	12
8.3. Ejemplos de uso de interfaces	12
8.3.1. Verificación pública por folio	12
8.3.2. Inicio de sesión	12
9. Flujos y Lógica de Negocio	14
9.1. Procesos clave del sistema	14

9.2. Flujo completo de documento	14
9.3. Reglas importantes	15
9.4. Casos críticos	15
10. Ejemplos de Uso	16
10.1. Configuración inicial de un usuario firmante	16
10.2. Firma de un documento	16
10.3. Ejecución de funciones clave	16
11. Manejo de Errores	17
11.1. Estrategia de errores	17
11.2. Mensajes comunes	17
11.3. Registro (logs)	17
12. Pruebas	18
12.1. Tipos de pruebas	18
12.2. Cómo ejecutarlas	18
13. Despliegue	20
13.1. Cómo hacerlo	20
13.2. Entornos	20
13.3. Consideraciones	20
14. Seguridad	21
14.1. Autenticación	21
14.2. Manejo de datos sensibles	21
14.3. Buenas prácticas implementadas	21
14.4. Riesgos y mitigaciones recomendadas	21
15. Guía de Contribución	22
15.1. Convenciones de código	22
15.2. Flujo de trabajo con Git	22
15.3. Cómo agregar funcionalidades	22
16. Problemas Conocidos	23
16.1. Bugs actuales	23
16.2. Limitaciones	23
17. FAQ Técnica	24
Referencias	24

Índice de tablas

1.1. Casos de uso principales del sistema (elaboración propia).	2
2.1. Componentes principales de la arquitectura (elaboración propia).	3
3.1. Stack tecnológico utilizado (elaboración propia).	5
4.1. Descripción de módulos del proyecto (elaboración propia).	6
5.1. Requisitos de entorno de desarrollo (elaboración propia).	7
5.2. Archivos de configuración relevantes (elaboración propia).	8
6.1. Comandos principales del proyecto (elaboración propia).	9
7.1. Descripción de tablas principales (elaboración propia).	10
8.1. Interfaces principales del sistema (elaboración propia).	12
9.1. Casos críticos y respuesta del sistema (elaboración propia).	15
11.1. Mensajes de error comunes (elaboración propia).	17
12.1. Matriz de validaciones y pruebas de integración (elaboración propia).	18
13.1. Entornos de despliegue (elaboración propia).	20
14.1. Riesgos de seguridad y mitigaciones recomendadas (elaboración propia).	21

Índice de figuras

2.1. Diagrama de arquitectura del sistema basado en contenedores Docker (elaboración propia).	4
7.1. Diagrama entidad-relación de la base de datos (elaboración propia).	10
9.1. Diagrama de flujo operativo del Gestor de Identidades (elaboración propia).	14
12.1. Evidencia de validaciones criptográficas y pruebas de autenticación ejecutadas en el sistema (elaboración propia).	19

1. Descripción General

1.1 Propósito del sistema

Casa Monarca v2 es una plataforma web institucional desarrollada para el Programa de Atención a Migrantes y sus Familias “Casa Monarca”. Su propósito es gestionar documentos institucionales con garantías de autenticidad, integridad y trazabilidad mediante mecanismos criptográficos modernos. El sistema permite crear documentos, firmarlos digitalmente, revocarlos cuando pierden validez y verificarlos públicamente por medio de un folio.

El gestor responde a una necesidad operativa concreta: sustituir procesos manuales de emisión y validación documental por un flujo digital controlado. Antes, la validez de un documento dependía de firmas físicas, sellos o confirmaciones manuales. Con la solución implementada, la validez queda respaldada por firmas digitales ECDSA P-256 y hashes SHA-256, de manera que cualquier alteración posterior del contenido invalida la verificación.

1.2 Alcance

El alcance de esta primera versión incluye la administración del ciclo de vida de documentos, la gestión de usuarios y roles, la firma criptográfica desde el navegador, la verificación pública de documentos y la bitácora de auditoría. La plataforma está pensada para operar en un entorno controlado mediante Docker Compose, con servicios separados para aplicación web y base de datos.

El sistema sí permite:

- Registrar usuarios institucionales y asignarles roles.
- Crear documentos en estado borrador.
- Firmar documentos con ECDSA P-256 usando una frase mnemónica BIP39 de 12 palabras.
- Registrar la clave pública del usuario en el servidor.
- Verificar documentos emitidos mediante folio público.
- Revocar documentos emitidos.
- Registrar acciones relevantes en bitácora con usuario, fecha, IP y resultado.

Límites de la versión actual:

- No implementa todavía carga completa de archivos adjuntos, aunque existe un campo reservado para ello en la base de datos.
- No cuenta aún con autenticación multifactor.
- No implementa rate limiting en el login.
- La limpieza periódica de sesiones de firma debe automatizarse en versiones posteriores.
- El endpoint de rotación de llaves del modo legado RSA aparece como pendiente de implementación.

1.3 Casos de uso principales

Los casos de uso principales se concentran en la operación documental, la administración de usuarios y la verificación externa de documentos.

Tabla 1.1: Casos de uso principales del sistema (elaboración propia).

ID	Caso de uso	Descripción
CU-01	Iniciar sesión	El usuario ingresa con correo y contraseña institucional. El sistema valida credenciales cifradas y crea sesión segura.
CU-02	Crear documento	Un usuario autorizado registra título, descripción y contenido. El documento queda en estado borrador.
CU-03	Firmar documento	Un usuario con permiso introduce su mnemónico BIP39; el navegador deriva la clave privada y firma el hash del documento.
CU-04	Revocar documento	Un administrador o supervisor invalida un documento emitido y registra el motivo de revocación.
CU-05	Verificar documento	Un tercero consulta un folio público para confirmar si el documento existe, está emitido y tiene firma válida.
CU-06	Gestionar usuarios	El administrador crea usuarios, cambia roles y desactiva cuentas cuando sea necesario.
CU-07	Consultar bitácora	Usuarios autorizados revisan eventos de autenticación, emisión, revocación y errores.

2. Arquitectura del Sistema

2.1 Tipo de arquitectura

La solución utiliza una arquitectura cliente-servidor modular y containerizada. La capa cliente se ejecuta en el navegador y contiene la interfaz de usuario junto con operaciones criptográficas críticas, como la derivación de llaves y la firma ECDSA P-256. La capa servidor, implementada con PHP y Apache, gestiona autenticación, sesiones, reglas de negocio, persistencia y verificación de firmas. La base de datos MySQL almacena usuarios, roles, documentos, claves públicas, bitácora y sesiones de firma.

El despliegue se realiza con Docker Compose, lo que permite levantar el sistema completo en un entorno reproducible. Esta decisión reduce la dependencia de proveedores externos y permite que Casa Monarca conserve control sobre su infraestructura y sus datos.

2.2 Componentes principales

Tabla 2.1: Componentes principales de la arquitectura (elaboración propia).

Componente	Responsabilidad
Cliente web	Presenta la interfaz, permite crear y consultar documentos, ingresar mnemónicos y ejecutar operaciones criptográficas del lado del navegador.
Servidor PHP + Apache	Atiende rutas de aplicación, valida sesiones, aplica reglas de negocio, consulta la base de datos y coordina operaciones de documentos y usuarios.
MySQL	Persiste usuarios, roles, documentos, claves públicas, bitácora, tokens y sesiones de firma.
Módulo criptográfico del cliente	Genera llaves ECDSA P-256, deriva claves desde mnemónicos BIP39 y firma hashes SHA-256 sin exponer la clave privada al servidor.
Bitácora	Registra acciones relevantes del sistema, incluyendo accesos, emisiones, revocaciones, errores de firma y cambios de estado.
Verificación pública	Permite validar documentos mediante folio sin requerir autenticación.

2.3 Diagrama general

En la Fig. 2.1 se muestra la arquitectura basada en contenedores Docker. El servicio web se comunica con la base de datos dentro de una red controlada por Docker Compose, mientras que el usuario accede desde el navegador. El diseño permite separar la lógica web, la persistencia y los módulos criptográficos.

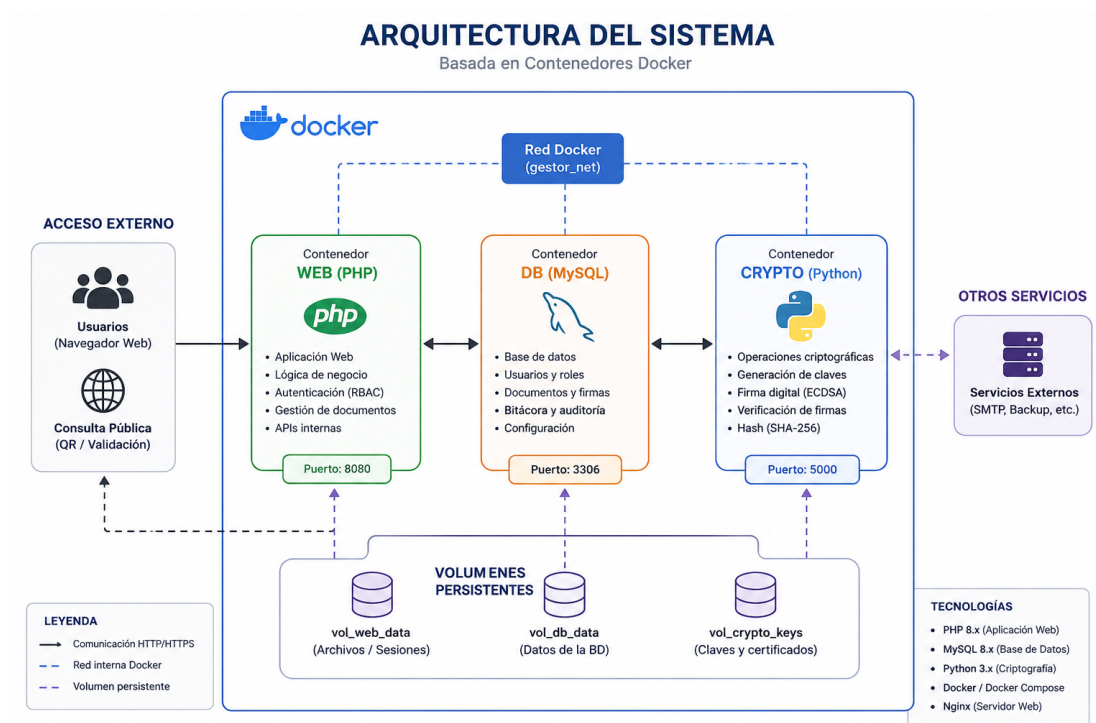


Fig. 2.1. Diagrama de arquitectura del sistema basado en contenedores Docker (elaboración propia).

3. Stack Tecnológico

Tabla 3.1: Stack tecnológico utilizado (elaboración propia).

Categoría	Tecnología	Versión / notas
Lenguaje backend	PHP	PHP 8.2 para lógica de servidor, sesiones y conexión con base de datos.
Servidor web	Apache	Apache 2.4 dentro del contenedor web.
Base de datos	MySQL	MySQL 8.0 para persistencia de usuarios, documentos, bitácora y sesiones.
Frontend	HTML5, CSS3, JavaScript	Interfaz web ligera con módulos ES6.
Criptografía cliente	@noble/curves	Biblioteca usada para operaciones ECDSA P-256 en navegador.
Hashing documental	SHA-256	Función hash aplicada al folio y contenido del documento antes de firmar.
Contraseñas	bcrypt	Hash irreversible de contraseñas con factor de costo configurado.
Contenedores	Docker / Docker Compose	Orquestación local del servidor web y la base de datos.
Control de versiones	Git / GitHub	Repositorio del equipo para control de cambios.

3.1 Justificación del stack

PHP, Apache y MySQL fueron seleccionados por su facilidad de despliegue, disponibilidad y compatibilidad con entornos institucionales. Docker Compose se utilizó para evitar instalaciones manuales de dependencias y asegurar que el sistema se ejecute de forma consistente en computadoras de desarrollo y servidores finales.

El uso de ECDSA P-256 en el navegador permite que la clave privada nunca abandone el dispositivo del firmante. El servidor almacena únicamente la clave pública, lo cual reduce el riesgo de compromiso de firmas históricas en caso de una intrusión al servidor.

4. Estructura del Proyecto

4.1 Organización de carpetas

La siguiente estructura resume una organización esperada del proyecto con base en los módulos descritos en el reporte técnico. Los nombres pueden variar ligeramente según el repositorio, pero representan la separación lógica entre aplicación, configuración, base de datos y documentación.

```
1 casa-monarca-v2/
2 |-- docker-compose.yml
3 |-- Dockerfile
4 |-- README.md
5 |-- public/
6 |   |-- index.php
7 |   |-- login.php
8 |   |-- dashboard.php
9 |   |-- documentos.php
10 |   |-- usuarios.php
11 |   |-- keygen.html
12 |   |-- verificar.html
13 |   '-- assets/
14 |       |-- css/
15 |       '-- js/
16 |-- src/
17 |   |-- db.php
18 |   |-- auth.php
19 |   |-- rbac.php
20 |   |-- audit.php
21 |   |-- documents.php
22 |   '-- crypto/
23 |-- database/
24 |   |-- schema.sql
25 |   '-- seed.sql
26 |-- tests/
27 |   |-- LoginTest.php
28 |   |-- RbacTest.php
29 |   '-- BitacoraTest.php
30 '-- docs/
31     |-- Manual_Usuario.pdf
32     |-- Reporte_Tecnico.pdf
33     '-- Documentacion_Desarrollo.tex
```

Listing 4.1: Estructura general del proyecto (elaboración propia).

4.2 Descripción de módulos

Tabla 4.1: Descripción de módulos del proyecto (elaboración propia).

Módulo	Descripción
public/	Contiene las vistas accesibles desde navegador, como login, dashboard, documentos, usuarios, llaves y verificación pública.
assets/js/	Contiene la lógica del cliente, incluyendo interacción de formularios y operaciones criptográficas en navegador.
src/db.php	Configura la conexión segura a MySQL mediante PDO y sentencias preparadas.
src/auth.php	Centraliza validación de sesión, login, logout y protección de rutas privadas.
src/rbac.php	Implementa validaciones de permisos por rol.
src/audit.php	Registra eventos relevantes en la bitácora de auditoría.
database/	Incluye scripts SQL para crear tablas, relaciones y datos iniciales.
tests/	Agrupar pruebas de autenticación, permisos, bitácora y validaciones funcionales.

5. Configuración del Entorno

5.1 Requisitos y versiones

Tabla 5.1: Requisitos de entorno de desarrollo (elaboración propia).

Herramienta	Versión mínima sugerida	Uso
Docker	24.x o superior	Ejecutar contenedores del sistema.
Docker Compose	3.8 o superior	Orquestar servicios web y base de datos.
PHP	8.2	Ejecutar la aplicación backend en el contenedor web.
Apache	2.4	Servidor HTTP de la aplicación.
MySQL	8.0	Base de datos relacional.
Navegador moderno	2022 o posterior	Ejecutar interfaz y criptografía del lado del cliente.
Git	2.x	Control de versiones y clonación del repositorio.

5.2 Variables de entorno

El sistema utiliza variables de entorno para separar credenciales y parámetros de configuración del código fuente. Un archivo `.env` local puede seguir la estructura siguiente:

```
1 # Aplicacion
2 APP_ENV=development
3 APP_URL=http://localhost:8080
4 APP_TIMEZONE=America/Monterrey
5
6 # Base de datos
7 DB_HOST=db
8 DB_PORT=3306
9 DB_NAME=casa_monarca
10 DB_USER=casa_user
11 DB_PASSWORD=cambiar_esta_password
12
13 # Seguridad
14 BCRYPT_COST=10
15 SESSION_COOKIE_HTTPONLY=true
16 SESSION_COOKIE_SAMESITE=Strict
```

Listing 5.1: Ejemplo de archivo `.env` para desarrollo (elaboración propia).

Advertencia

Nunca se debe subir el archivo `.env` al repositorio. Debe incluirse en `.gitignore` y compartirse únicamente mediante canales seguros.

5.3 Archivos de configuración

Tabla 5.2: Archivos de configuración relevantes (elaboración propia).

Archivo	Función
<code>docker-compose.yml</code>	Define servicios, red interna, puertos y volúmenes persistentes.
<code>Dockerfile</code>	Construye la imagen del servicio web con PHP y Apache.
<code>database/schema.sql</code>	Crea la estructura de tablas y relaciones.
<code>database/seed.sql</code>	Inserta datos iniciales, como roles o usuario administrador.
<code>src/db.php</code>	Lee variables de entorno y crea la conexión PDO.

6. Instalación y Ejecución

6.1 Pasos para levantar el sistema

Paso 1. Clonar el repositorio del proyecto:

```
1 git clone https://github.com/N1c0l4s-RF/casa-monarca-v2.git
2 cd casa-monarca-v2
```

Paso 2. Crear el archivo de variables de entorno a partir del ejemplo del equipo:

```
1 cp .env.example .env
2 # Editar usuario, password y nombre de base de datos segun el entorno
```

Paso 3. Construir y levantar los contenedores:

```
1 docker compose up --build
```

Paso 4. Acceder al sistema desde el navegador:

```
1 http://localhost:8080
```

Paso 5. Iniciar sesión con las credenciales institucionales o con el usuario administrador configurado en el entorno de pruebas.

6.2 Comandos principales

Tabla 6.1: Comandos principales del proyecto (elaboración propia).

Comando	Descripción
<code>docker compose up -build</code>	Construye imágenes y levanta todos los servicios.
<code>docker compose up -d</code>	Levanta los servicios en segundo plano.
<code>docker compose down</code>	Detiene y elimina contenedores de la aplicación.
<code>docker compose logs -f web</code>	Muestra logs del servicio web en tiempo real.
<code>docker compose logs -f db</code>	Muestra logs de la base de datos.
<code>docker compose exec db mysql -u root -p</code>	Abre una consola MySQL dentro del contenedor de base de datos.

7. Base de Datos

7.1 Modelo de datos

El modelo de datos se centra en usuarios, roles, documentos, claves públicas, bitácora y sesiones de firma. Cada documento tiene un creador y, cuando es emitido, un firmante. La bitácora conserva evidencia de acciones relevantes, mientras que las claves públicas permiten verificar firmas sin almacenar claves privadas.

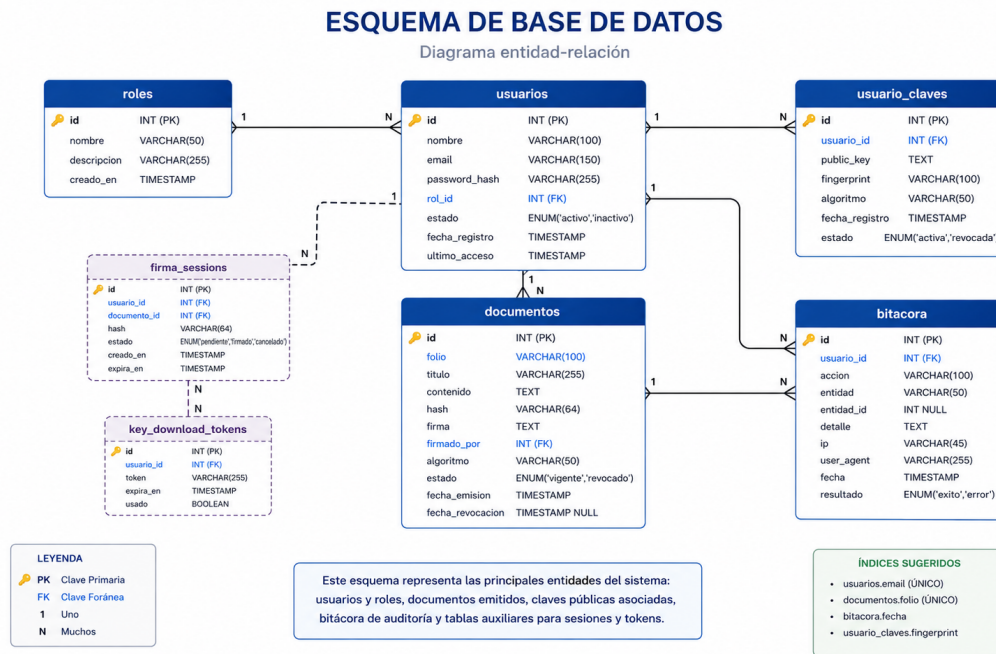


Fig. 7.1. Diagrama entidad-relación de la base de datos (elaboración propia).

7.2 Tablas principales

Tabla 7.1: Descripción de tablas principales (elaboración propia).

Tabla	Campo clave	Descripción
roles	id	Catálogo de roles del sistema y su descripción operativa.
usuarios	id	Información de cuentas institucionales, correo, contraseña cifrada, rol y estado.
usuario_claves	id	Guarda claves públicas registradas por usuario, fingerprint y algoritmo.
documentos	id, folio	Registra documentos, estado, contenido, creador, firmante, hash y firma digital.
bitacora	id	Almacena eventos de auditoría con acción, resultado, IP, fecha y usuario asociado.
firma_sessions	id	Controla sesiones temporales asociadas al flujo de firma.

Tabla	Campo clave	Descripción
key_download_tokens	id	Apoya operaciones temporales de recuperación o descarga de material criptográfico cuando aplica.

7.3 Relaciones

- Un rol puede estar asociado a muchos usuarios: relación `roles` 1:N `usuarios`.
- Un usuario puede crear muchos documentos: relación `usuarios` 1:N `documentos` mediante el campo de creador.
- Un usuario puede firmar muchos documentos: relación `usuarios` 1:N `documentos` mediante el campo de firmante.
- Un usuario puede tener una o varias claves públicas registradas dependiendo del historial de recuperación o actualización de llaves.
- Cada evento de bitácora puede asociarse a un usuario y, cuando aplica, a un documento.

8. API / Interfaces

8.1 Convenciones generales

La aplicación opera principalmente mediante páginas PHP y formularios web protegidos por sesión. Las operaciones privadas requieren autenticación y validación de rol. Las consultas públicas de verificación se exponen sin cuenta de usuario, pero limitan la información mostrada al estado del documento y datos mínimos de validación.

Las respuestas internas pueden ser HTML o JSON según el flujo, mientras que los endpoints asociados a acciones suelen recibir parámetros por POST. Todas las consultas a base de datos deben ejecutarse mediante PDO y sentencias preparadas.

8.2 Interfaces principales

Tabla 8.1: Interfaces principales del sistema (elaboración propia).

Ruta / interfaz	Método	Descripción
/login.php	GET / POST	Muestra formulario de acceso y valida credenciales.
/logout.php	POST / GET	Cierra la sesión activa e invalida el acceso del usuario.
/dashboard.php	GET	Muestra resumen de documentos, contadores y accesos principales.
/documentos.php	GET / POST	Lista documentos, permite crear borradores, emitir y revocar según permisos.
/usuarios.php	GET / POST	Gestiona usuarios, roles y desactivación de cuentas.
/keygen.html	GET	Interfaz independiente para generar, recuperar o registrar llaves criptográficas.
/verificar.html	GET	Interfaz pública para consultar folios documentales.
/documentos-verificar.php	GET	Procesa la verificación pública de un documento por folio.
/consulta_qr.php	GET	Ruta de consulta orientada a verificación mediante código QR cuando aplica.

8.3 Ejemplos de uso de interfaces

8.3.1 Verificación pública por folio

```
1 GET /documentos-verificar.php?folio=DOC-20260512-ABC123
```

Listing 8.1: Ejemplo conceptual de consulta pública por folio.

Resultado esperado: el sistema muestra si el documento existe, su estado, el firmante, la fecha de emisión, el algoritmo usado y la firma asociada. Si el documento no existe o fue revocado, se muestra el estado correspondiente.

8.3.2 Inicio de sesión

```
1 POST /login.php
2 Content-Type: application/x-www-form-urlencoded
3
4 email=usuario@casamonarca.org&password=*****
```

Listing 8.2: Ejemplo conceptual de envío de credenciales.

Resultado esperado: si las credenciales son válidas y el usuario está activo, se crea una sesión segura y se redirige al dashboard.

9. Flujos y Lógica de Negocio

9.1 Procesos clave del sistema

La lógica operativa del sistema parte de la autenticación, continúa con validación de rol y termina con el registro de la acción en bitácora. La Fig. 9.1 muestra el flujo general del gestor desde el acceso hasta el cierre de operación.

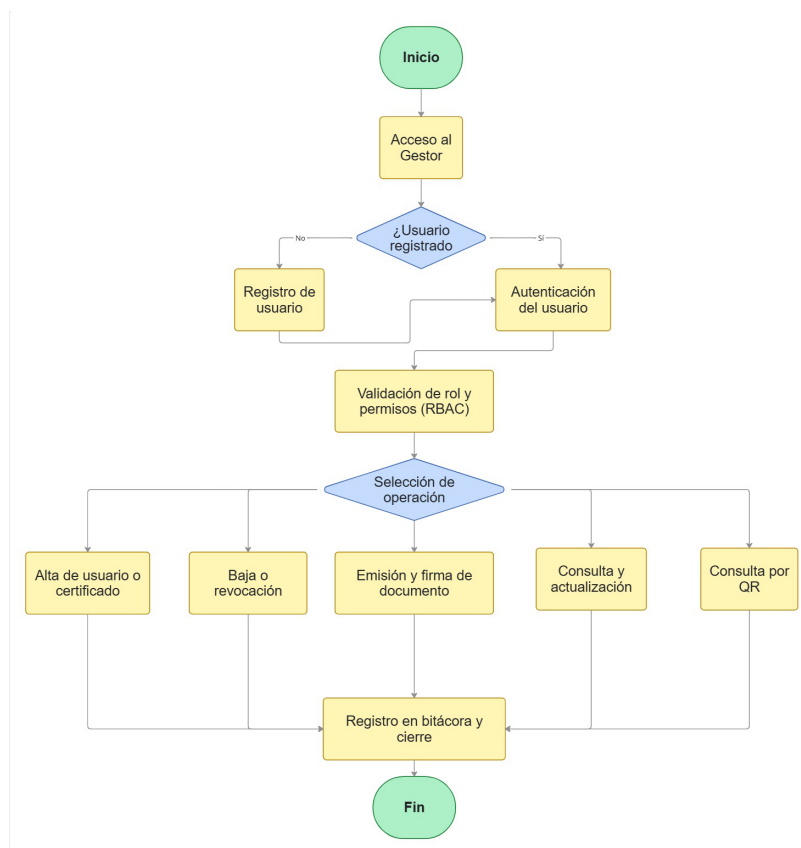


Fig. 9.1. Diagrama de flujo operativo del Gestor de Identidades (elaboración propia).

9.2 Flujo completo de documento

1. Un usuario autorizado crea un documento con título, descripción y contenido.
2. El sistema genera un folio único con formato DOC-YYYYMMDD-XXXXXX.
3. El documento queda en estado *borrador*.
4. Un usuario con rol Administrador, Supervisor o Emisor firma el documento, siempre que cumpla la regla de segregación de funciones.
5. El navegador deriva la clave privada a partir del mnemónico BIP39 y firma el hash SHA-256 del documento.
6. El servidor recibe la firma, valida la clave pública registrada y cambia el estado a *emitido*.
7. La operación se registra en bitácora.
8. Cualquier tercero puede verificar el documento con su folio público.

9.3 Reglas importantes

- La clave privada nunca se almacena en el servidor ni viaja por la red.
- Los documentos en borrador deben ser firmados por un usuario distinto al creador, excepto cuando el rol Administrador tenga permiso especial.
- Solo Administrador y Supervisor pueden revocar documentos emitidos.
- Las contraseñas se almacenan mediante bcrypt, nunca en texto plano.
- Las consultas públicas no deben revelar información sensible innecesaria.
- Toda acción crítica debe generar un registro de auditoría.

9.4 Casos críticos

Tabla 9.1: Casos críticos y respuesta del sistema (elaboración propia).

Caso crítico	Respuesta esperada
Usuario intenta firmar su propio documento	El sistema bloquea la operación por segregación de funciones, salvo excepción de administrador.
Mnemónico incorrecto	La firma no coincide con la clave pública registrada y se registra intento fallido.
Documento alterado después de emitirse	El hash cambia y la verificación criptográfica falla.
Usuario desactivado intenta acceder	Se rechaza la autenticación o se bloquea el acceso a funciones privadas.
Folio inexistente	La verificación pública informa que el documento no fue encontrado.
Documento revocado	La verificación pública muestra que el documento ya no es válido.

10. Ejemplos de Uso

10.1 Configuración inicial de un usuario firmante

1. El administrador crea la cuenta del usuario desde el módulo *Usuarios*.
2. El usuario inicia sesión con sus credenciales institucionales.
3. Ingresa a *Mis Llaves* o a `/keygen.html`.
4. Genera 12 palabras mnemónicas en español.
5. Las anota en papel y confirma que las guardó.
6. Registra su clave pública en el servidor.
7. A partir de ese momento puede firmar documentos según su rol.

10.2 Firma de un documento

```
1 1. Usuario A crea documento en borrador.  
2 2. Usuario B abre el documento y selecciona Emitir.  
3 3. Usuario B ingresa sus 12 palabras BIP39.  
4 4. El navegador deriva la clave privada ECDSA P-256.  
5 5. Se calcula SHA-256(folio | contenido).  
6 6. Se firma el hash y se envía la firma al servidor.  
7 7. El servidor verifica la firma con la clave publica registrada.  
8 8. El documento pasa a estado emitido.
```

Listing 10.1: Flujo conceptual de firma de documento.

10.3 Ejecución de funciones clave

El siguiente fragmento representa, de forma conceptual, la validación de acceso basada en roles antes de ejecutar una acción sensible. No sustituye el código final del repositorio, pero documenta la lógica esperada.

```
1 function requireRole(array $rolesPermitidos, string $rolUsuario): void {  
2     if (!in_array($rolUsuario, $rolesPermitidos, true)) {  
3         registrarBitacora('acceso_denegado', 'fallido');  
4         http_response_code(403);  
5         exit('No tienes permiso para realizar esta operacion.');6     }  
7 }  
8  
9 requireRole(['administrador', 'supervisor', 'emisor'], $usuario['rol']);
```

Listing 10.2: Validación conceptual de permisos por rol (elaboración propia).

11. Manejo de Errores

11.1 Estrategia de errores

El sistema debe manejar errores de forma segura, evitando revelar detalles internos de la base de datos, rutas del servidor o información sensible. Los errores se clasifican en autenticación, permisos, validación de datos, operaciones criptográficas, base de datos y disponibilidad del servicio.

Las acciones fallidas relevantes deben registrarse en bitácora con su resultado, dirección IP, usuario cuando exista sesión activa y descripción general del evento. En la interfaz, el usuario debe recibir mensajes claros y accionables, sin exponer trazas internas.

11.2 Mensajes comunes

Tabla 11.1: Mensajes de error comunes (elaboración propia).

Código	Mensaje	Causa / solución
400	Solicitud inválida	Faltan campos obligatorios o el formato del folio no es correcto. Revisar datos enviados.
401	Credenciales inválidas	Correo o contraseña incorrectos. Verificar credenciales institucionales.
403	Acceso denegado	El rol del usuario no tiene permisos para ejecutar la operación solicitada.
404	Documento no encontrado	El folio consultado no existe o no corresponde a un documento registrado.
409	Operación no permitida	El usuario intenta firmar su propio documento o modificar un estado inválido.
422	Firma inválida	La firma no corresponde al hash del documento o a la clave pública registrada.
500	Error interno	Fallo del servidor o base de datos. Revisar logs del contenedor web y de MySQL.

11.3 Registro (logs)

Los logs operativos se consultan mediante Docker Compose. Para depuración se recomienda revisar primero el servicio web y después la base de datos.

```
1 docker compose logs -f web
2 docker compose logs -f db
```

Listing 11.1: Consulta de logs del sistema.

Además de los logs de contenedor, el sistema mantiene una bitácora funcional dentro de la base de datos. Esta bitácora permite auditar eventos como login, logout, documentos creados, documentos emitidos, revocaciones, claves registradas y firmas inválidas.

12. Pruebas

12.1 Tipos de pruebas

Las pruebas documentadas se enfocan en validar los módulos críticos del sistema: autenticación, permisos, integridad documental, firma digital, trazabilidad y consulta pública.

Tabla 12.1: Matriz de validaciones y pruebas de integración (elaboración propia).

Caso de uso	Validación técnica	Resultado esperado	Evidencia
Autenticación	Verificación de credenciales cifradas y manejo de sesiones.	Acceso permitido únicamente a usuarios válidos.	LoginTest.php
Seguridad Login	Intento de acceso con usuario inválido o deshabilitado.	Bloqueo de autenticación y mensaje de error.	LoginTest.php
Control RBAC	Validación de permisos según rol operativo.	Restricción de operaciones no autorizadas.	RbacTest.php
Integridad documental	Generación de hash SHA-256.	Huella digital única y verificable.	Módulo ECDSA
Firma ECDSA	Generación y validación de firmas P-256.	Firma criptográfica válida y verificable.	Módulo de llaves
Trazabilidad	Registro automático de operaciones.	Inserción correcta de usuario, IP y timestamp.	BitacoraTest.php
Consulta pública	Verificación mediante folio o QR.	Visualización del estado sin autenticación.	consulta_qr.php

12.2 Cómo ejecutarlas

La forma exacta de ejecución depende de la organización final del repositorio. En un entorno basado en contenedores, las pruebas pueden ejecutarse desde el contenedor web o desde el ambiente local con PHP instalado.

```
1 # Entrar al contenedor web
2 docker compose exec web bash
3
4 # Ejecutar pruebas PHP si el proyecto usa PHPUnit
5 vendor/bin/phpunit tests/
6
7 # Alternativa conceptual por archivo
8 php tests/LoginTest.php
9 php tests/RbacTest.php
10 php tests/BitacoraTest.php
```

Listing 12.1: Comandos sugeridos para pruebas.

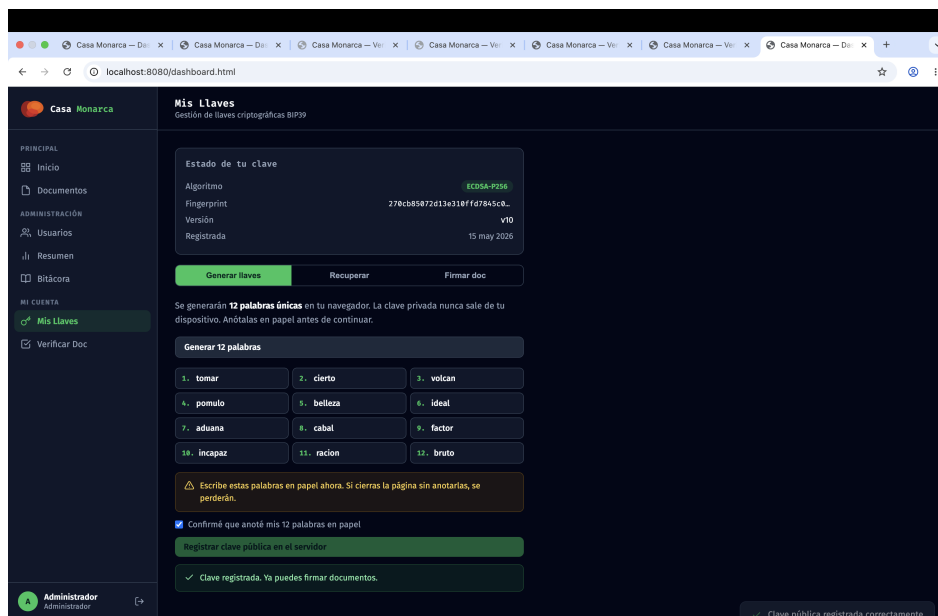


Fig. 12.1. Evidencia de validaciones criptográficas y pruebas de autenticación ejecutadas en el sistema (elaboración propia).

13. Despliegue

13.1 Cómo hacerlo

El despliegue recomendado para esta versión es mediante Docker Compose. Este enfoque permite levantar la aplicación y la base de datos con un solo comando, manteniendo volúmenes persistentes para datos y configuraciones.

1. Preparar el servidor con Docker y Docker Compose.
2. Clonar el repositorio del proyecto.
3. Configurar el archivo `.env` con credenciales seguras.
4. Ejecutar `docker compose up -build -d`.
5. Verificar que el servicio web responda en el puerto configurado.
6. Crear o validar el usuario administrador inicial.
7. Probar login, creación de documento, firma y verificación pública.

13.2 Entornos

Tabla 13.1: Entornos de despliegue (elaboración propia).

Entorno	URL	Rama	Notas
Desarrollo	<code>http://localhost:8080</code>	<code>dev</code> o rama local	Usado para pruebas del equipo.
Piloto	Red interna de Casa Monarca	<code>main</code>	Recomendado para validación controlada.
Producción	Dominio institucional	<code>main</code>	Debe incluir HTTPS, respaldos y monitoreo.

13.3 Consideraciones

Nota

Para producción se recomienda usar HTTPS, contraseñas robustas, respaldos automáticos de MySQL, monitoreo de logs, limpieza programada de sesiones expiradas y control de acceso al servidor. También se recomienda evitar hosting compartido si impide controlar versiones, servicios, puertos o persistencia de datos.

14. Seguridad

14.1 Autenticación

La autenticación se basa en correo institucional y contraseña. Las contraseñas se almacenan como hashes bcrypt irreversibles, lo que impide recuperar la contraseña original incluso desde la base de datos. Las sesiones usan cookies con propiedades de seguridad como `HttpOnly` y `SameSite=Strict` cuando el entorno lo permite.

14.2 Manejo de datos sensibles

El dato más sensible del sistema es la clave privada del firmante. Por diseño, esta clave no se almacena en el servidor. Se deriva localmente en el navegador a partir de una frase mnemónica BIP39 de 12 palabras. El servidor conserva únicamente la clave pública comprimida, que sirve para verificar firmas.

La validación documental se realiza con SHA-256 sobre el folio y contenido del documento. Después, el hash se firma usando ECDSA P-256. Si el contenido cambia, el hash cambia y la firma deja de coincidir.

14.3 Buenas prácticas implementadas

- Firma digital del lado del cliente para evitar custodia centralizada de claves privadas.
- Uso de bcrypt para contraseñas.
- Control de acceso basado en roles.
- Segregación de funciones entre creador y firmante.
- Bitácora de auditoría inmutable desde la interfaz.
- Consultas SQL mediante sentencias preparadas.
- Mínima exposición de datos en verificación pública.
- Despliegue reproducible con Docker Compose.

14.4 Riesgos y mitigaciones recomendadas

Tabla 14.1: Riesgos de seguridad y mitigaciones recomendadas (elaboración propia).

Riesgo	Mitigación recomendada
Pérdida de mnemónico	Capacitar a usuarios para guardar las 12 palabras en papel y definir protocolo institucional de recuperación de identidad.
Fuerza bruta en login	Implementar rate limiting por IP y por cuenta, con ventanas de tiempo y bloqueo temporal.
Sesiones expiradas acumuladas	Automatizar limpieza de <code>firma_sessions</code> mediante cron o eventos MySQL.
Acceso indebido al servidor	Usar HTTPS, firewall, claves SSH y mínimo privilegio en usuarios del sistema.
Falta de MFA	Añadir TOTP o WebAuthn para roles de alta criticidad.

15. Guía de Contribución

15.1 Convenciones de código

- Usar nombres descriptivos para funciones, variables y archivos.
- Separar lógica de base de datos, autenticación, permisos y vistas.
- Evitar credenciales en el código fuente.
- Documentar funciones críticas, especialmente las relacionadas con firma, verificación y permisos.
- Mantener validaciones tanto en frontend como en backend.
- Registrar en bitácora toda operación sensible.

15.2 Flujo de trabajo con Git

```
1 # 1. Actualizar rama principal
2 git checkout main
3 git pull origin main
4
5 # 2. Crear rama de funcionalidad
6 git checkout -b feature/nombre-funcionalidad
7
8 # 3. Hacer commits descriptivos
9 git add .
10 git commit -m "feat: descripcion breve del cambio"
11
12 # 4. Subir rama
13 git push origin feature/nombre-funcionalidad
14
15 # 5. Abrir Pull Request hacia main o dev
```

Listing 15.1: Flujo de trabajo sugerido con Git.

15.3 Cómo agregar funcionalidades

1. Identificar el módulo afectado: documentos, usuarios, llaves, bitácora o verificación.
2. Crear una rama nueva de desarrollo.
3. Implementar la funcionalidad con validaciones de permisos.
4. Agregar pruebas o evidencia funcional.
5. Revisar que no se expongan datos sensibles.
6. Actualizar esta documentación si cambia la API, base de datos o flujo de usuario.
7. Solicitar revisión de otro integrante antes de integrar cambios.

16. Problemas Conocidos

16.1 Bugs actuales

Al tratarse de una primera versión funcional, no se documentan bugs bloqueantes dentro del flujo principal de creación, firma, emisión y verificación. Sin embargo, existen pendientes técnicos y mejoras identificadas para una versión de producción.

16.2 Limitaciones

- El endpoint de rotación de llaves RSA del modo legado está pendiente y puede responder como no implementado.
- No existe rate limiting en el endpoint de login.
- El campo `ruta_archivo` está reservado para adjuntos, pero la lógica completa de carga de archivos queda pendiente.
- La limpieza periódica de sesiones de firma debe automatizarse.
- La verificación por QR debe alinearse completamente con el flujo ECDSA para evitar diferencias entre rutas de verificación.
- En hosting compartido pueden existir restricciones de puertos, versiones de PHP, acceso a Docker, tareas programadas y control sobre MySQL.

17. FAQ Técnica

¿Por qué no levanta el servidor al ejecutar Docker Compose?

Revisa que Docker esté abierto, que los puertos configurados no estén ocupados y que el archivo `.env` exista. Después ejecuta `docker compose logs -f web` para revisar el error.

¿Por qué no conecta con MySQL?

Verifica que `DB_HOST` coincida con el nombre del servicio de base de datos definido en `docker-compose.yml`. En Docker normalmente debe usarse el nombre del servicio, por ejemplo `db`, no `localhost`.

¿Qué pasa si un usuario pierde sus 12 palabras?

No puede recuperar la misma clave privada. Debe generar un nuevo par de llaves y solicitar al administrador la actualización de su registro. Los documentos firmados previamente siguen asociados a la clave pública anterior.

¿Por qué un usuario no puede firmar cierto documento?

Puede deberse a que no tiene rol autorizado, a que él mismo creó el documento, a que el documento no está en borrador o a que no ha registrado su clave pública.

¿Cómo se verifica si un documento fue alterado?

El servidor recalcula el hash SHA-256 del folio y contenido. Si el contenido cambió, la firma ECDSA ya no coincide con la clave pública del firmante y la verificación falla.

¿Dónde se revisan las acciones realizadas?

En el módulo de Bitácora, donde se registran accesos, documentos creados, firmas, revocaciones, errores y eventos relevantes.

Referencias

- [1] National Institute of Standards and Technology. (2023). *FIPS PUB 186-5: Digital Signature Standard (DSS)*. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.186-5>
- [2] National Institute of Standards and Technology. (2015). *FIPS PUB 180-4: Secure Hash Standard (SHS)*. U.S. Department of Commerce.
- [3] Wuille, P., Maxwell, G., et al. (2013). *BIP-0039: Mnemonic code for generating deterministic keys*. Bitcoin Improvement Proposals. <https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki>
- [4] Docker Inc. (2026). *Docker Documentation*. <https://docs.docker.com/>
- [5] The PHP Group. (2026). *PHP Manual*. <https://www.php.net/manual/>
- [6] Oracle. (2026). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>
- [7] Equipo 3. (2026). *Repositorio Casa Monarca v2*. <https://github.com/N1c0l4s-RF/casa-monarca-v2>